

DEEP LEARNING · VIVA PRESENTATION

Fashion-MNIST Image Classification Using Simple RNN

An end-to-end baseline for 10-class clothing classification

Made with GAMMA

01 · MOTIVATION

Why classify clothing images with an RNN?

Build the workflow

Build an end-to-end deep learning workflow for 10-class clothing classification.

Read rows as a sequence

Explore how a recurrent model can process image rows as a sequence.

Learn from the baseline

Compare the educational value of a SimpleRNN approach with image-native models.

Made with GAMMA

02 · DATA REPRESENTATION

Fashion-MNIST turns each image into a sequence

A 28 × 28 grayscale image becomes 28 time steps, with 28 pixel features available at every step.

- **70,000 images**

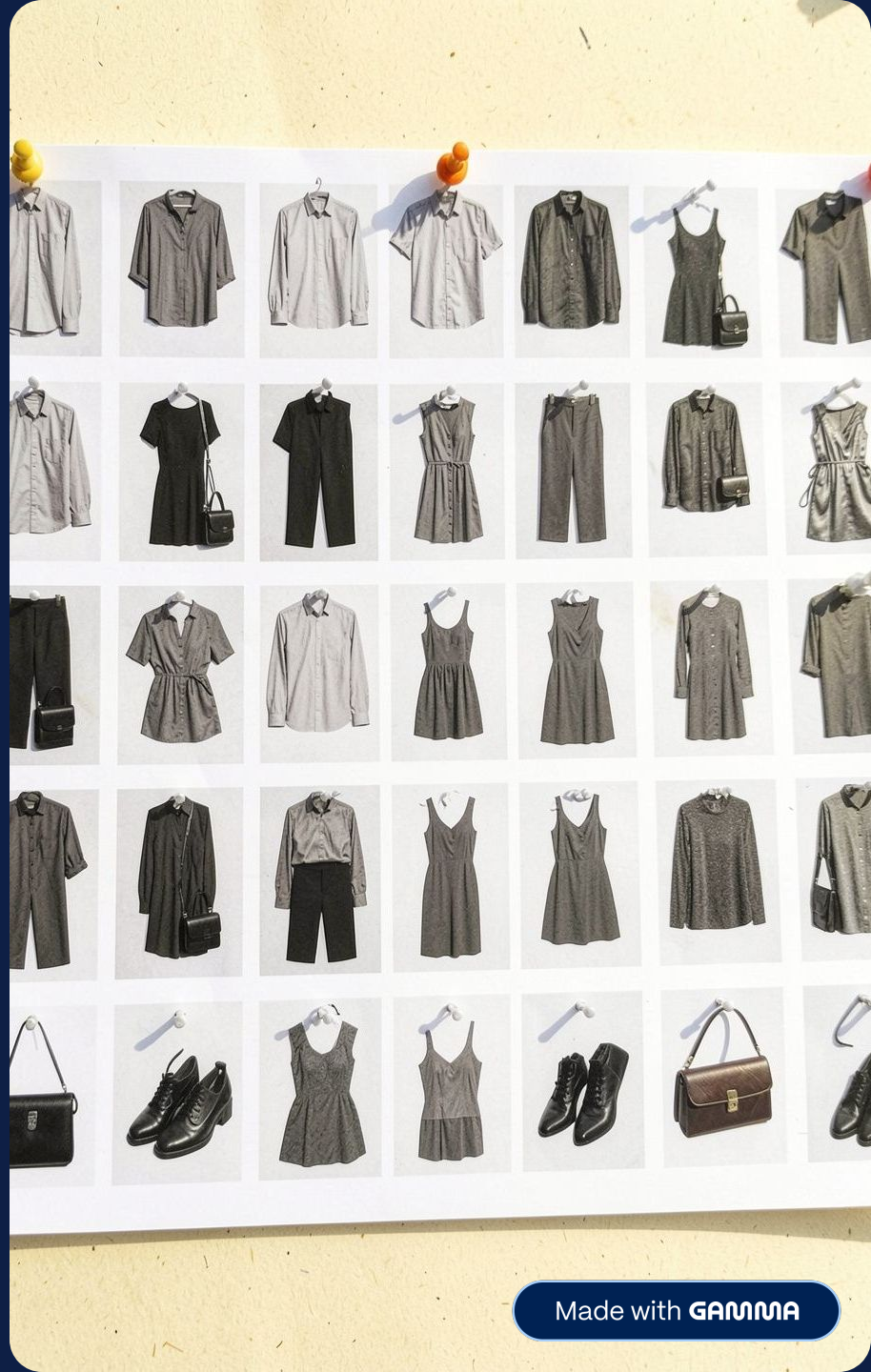
  Grayscale clothing images, each 28 × 28 pixels.

- **60,000 + 10,000**

  60,000 images are used for development; 10,000 official test images remain held out.

- **10 classes**

  T-shirt/top, Trouser, Pullover, Dress, Coat, Sandal, Shirt, Sneaker, Bag, and Ankle boot.



Made with GAMMA

03 · PREPROCESSING

Preprocessing makes the data model-ready

The input keeps its row structure while the values and split become consistent for training.

01

Normalize pixels

Normalize pixel values from 0–255 to 0–1 by dividing by 255.0.

02

Shape the sequence

Represent each 28 × 28 image as 28 time steps with 28 features per step.

03

Split the training set

Use a stratified 90/10 split: 54,000 training images and 6,000 validation images.

Made with GAMMA

04 · MODEL ARCHITECTURE

Two recurrent layers learn patterns across image rows

1

SimpleRNN: 128 units

`return_sequences=True`

2

Dropout 0.2

SimpleRNN: 64 units

3

Dropout 0.3

Dense: 64 units with ReLU; Dropout 0.2.

4

Softmax output

10 class probabilities.

Made with GAMMA

05 · TRAINING DESIGN

Training balances learning speed and generalization

Optimizer

Adam optimizer with learning rate 0.001.

Objective

Sparse categorical cross-entropy loss and accuracy tracking.

Training budget

Up to 20 epochs with batch size 128.

Adaptive controls

EarlyStopping restores the best weights; ReduceLROnPlateau lowers the learning rate when validation loss stalls.

Made with GAMMA

06 · EVALUATION

Evaluation asks more than “how accurate?”

Overall performance

Test accuracy and test loss measure performance on unseen images.

Class-level performance

Precision, recall, and F1-score reveal class-level strengths and weaknesses.

Error structure

A confusion matrix shows which clothing categories are confused most often.

Training behaviour

Learning curves help identify convergence or overfitting.

Made with GAMMA

07 · ACTUAL RUN REQUIRED

Results must come from the actual Colab run

No performance value is assumed. Replace the blanks after executing the notebook.

Metric	Colab result
Test accuracy	______
Test loss	______
Weighted precision	______
Weighted recall	______
Weighted F1-score	______

Interpret only after the notebook has produced the metrics.

Made with GAMMA

08 · PROJECT OUTPUTS

The workflow produces reusable project artifacts

Training evidence

Training and validation accuracy/loss graphs.

Evaluation evidence

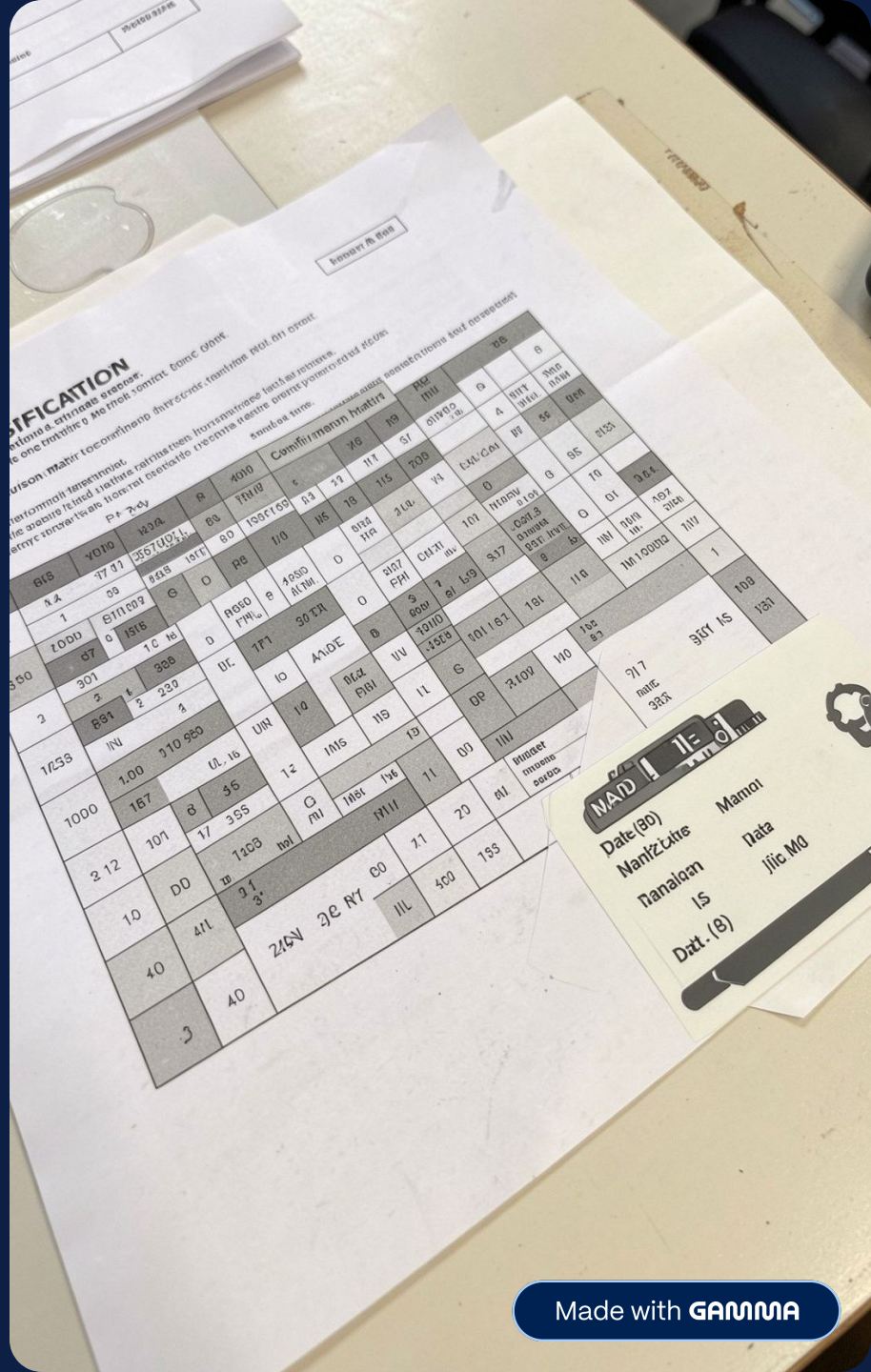
Confusion matrix, classification report, and example predictions.

Prediction file

fashion_mnist_rnn_predictions.csv with labels, predictions, correctness, and confidence.

Saved model

fashion_mnist_simple_rnn.keras for later inference.



Made with GAMMA

09 · TAKEAWAY

The SimpleRNN baseline teaches us how the full pipeline behaves.

- **End to end**

  It demonstrates the full pipeline from dataset loading to saved predictions.

- **Structural limitation**

  Row sequences do not capture two-dimensional image structure as naturally as a CNN.

- **Run sensitivity**

  Performance can vary with initialization, hardware, training time, and library versions.

- **Next comparison**

  Compare fairly against a CNN and test a CNN-RNN hybrid.

Made with GAMMA